

Course Outline: Agent Skills — Teaching Your Agent New Abilities

Title: Agent Skills — Teaching Your Agent New Abilities **Learnpack:** + Skills: ¿Cómo puede tu agente aprender habilidades casi como Neo en Matrix? **Prerequisite:** Memory bank learnpack (earlier Day 14) + Rules & Spec-Driven Design (Day 12) **Target Audience:** Beginner developers already familiar with coding agent rules, specifications, and memory banks — learning how to extend agent capabilities through skills. **Total Lessons:** 11 **Format:** Conceptual (0% hands-on)

Course Description

By this point in the bootcamp, students have learned to communicate with coding agents using specifications, configure rules to shape agent behavior, and persist knowledge through memory banks. Each of these is a powerful lever — but they share a limitation: they operate as disconnected pieces. Rules tell the agent what to follow. Memory gives it persistence. Specs frame a task. None of them package a complete, reusable *ability*.

This learnpack introduces **agent skills** — self-contained units that combine instructions, context, and purpose into something the agent can pick up and execute like a new capability. Think of it as the difference between giving someone a list of traffic laws (rules) versus teaching them how to drive (a skill). Students will understand what skills are, how they differ from rules and memory, how to add them to their coding agents, and where to find pre-built skills created by the community.

This is an understanding-focused learnpack. Students will learn the *what*, *why*, and *where* of agent skills. The next learnpack ("Creación de Skills") will dive into actually building skills from scratch — structure, quality criteria, composability, and testing.

Course Philosophy

- This course emphasizes:
- **Skills as context engineering:** A skill is not just an instruction — it's a curated package of context designed to make an agent effective at a specific task. This connects to the broader principle that the quality of AI output depends on the quality of its context.
 - **Evolution, not replacement:** Skills don't replace rules or memory. They build on top of them. Understanding when each tool is appropriate is more valuable than defaulting to any single approach.
 - **Path of least resistance:** Before creating custom skills, students should know that a rich ecosystem of pre-built skills already exists. Use what's available, adapt what's close, build only what's missing.
 - **Code is context too:** An agent's skills operate on a codebase. The quality of that codebase directly affects how well skills perform — good practices amplify, bad practices compound.
-

Course Statistics Summary

Section	Lessons	Estimated Words
---------	---------	-----------------

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - What Are Agent Skills?	2	~1,100
02 - Teaching Your Agent New Skills	2	~1,100
03 - Finding and Using Pre-Built Skills	2	~1,100
04 - Skills in Context	3	~1,900
05 - Conclusion	1	~450
Total	11	~6,100

Section 00: Welcome

00.0 Why Your Agent Needs More Than Rules (~450 words)

Learning Objectives:

- Recognize the limitations of rules alone when working with coding agents on complex tasks
- Articulate why a new abstraction (skills) becomes necessary as projects grow

Content Outline:

- Open with a familiar scenario: a student has set up rules for their coding agent — formatting rules, file structure rules, testing rules. The agent follows them. But when asked to perform a complex, multi-step task (e.g., "scaffold a new API endpoint following our project conventions"), the agent struggles. It has rules but no *ability*.
- The gap between "knowing the rules" and "being able to do something" — this is the same gap humans experience. Knowing grammar rules doesn't make you a writer.
- Brief preview: this learnpack will cover what skills are, how to give them to your agent, and where to find skills others have already built.
- Reference to what students already know: rules (Day 12), memory banks (earlier today). Skills are the next piece of the puzzle.

Transitions: Bridges from students' existing knowledge of rules and memory into the concept of skills. Sets up Section 01 by establishing the *need* before introducing the *solution*.

Key Principles:

- Rules define constraints; skills define capabilities
- Complexity in projects demands more than configuration — it demands packaged knowledge

Examples:

- A chef analogy: rules are like "always wash your hands" and "keep the kitchen clean." A skill is "how to make a béarnaise sauce" — it combines multiple steps, context, and judgment into one cohesive ability.
- A coding agent with perfect formatting rules that still can't scaffold a new feature because it doesn't know *how* your team structures features.

Section 01: What Are Agent Skills?

01.0 From Rules to Skills: What Changed and Why 📖 (~550 words)

Learning Objectives:

- Define what an agent skill is and how it differs from a rule
- Explain why skills represent an evolution of rules rather than a replacement
- Identify the key components that make a skill more than just a long rule

Content Outline:

- Start from what students know: rules are directives — "always use TypeScript," "never commit directly to main," "use Pydantic for validation." They shape behavior but don't teach capability.
- A skill is a **self-contained package** that gives an agent a specific ability: "how to create a new API endpoint in this project," "how to write a migration file," "how to generate a component following our design system."
- What makes a skill different from a rule:
 - **Purpose-driven:** A skill exists to accomplish something specific, not just to constrain behavior.
 - **Context-rich:** A skill carries the context needed to execute — not just the instruction, but the *why*, the *when*, and the *how*.
 - **Scoped:** A skill has clear boundaries — it knows what it does and what it doesn't do.
- The evolution metaphor: rules are like traffic laws. Skills are like knowing how to drive. You need both, but they serve different functions.
- Connection to the Thinking Framework: "Skills are an evolution of rules with a clearer purpose and context." This is not about discarding rules — it's about recognizing that some agent behaviors need richer packaging.

Transitions: Now that students understand *what* a skill is conceptually, the next lesson examines how skills actually work under the hood — what happens when an agent encounters a skill.

Key Principles:

- A skill bundles instruction + context + purpose into a single reusable unit
- Skills don't replace rules — they operate at a different level of abstraction
- The quality of a skill depends on the clarity of its purpose and the richness of its context

Examples:

- Rule: "Use snake_case for Python variables." Skill: "How to create a new FastAPI endpoint in this project, including the router file, Pydantic model, and test stub."
- Rule: "Always write tests." Skill: "How to write integration tests for our API using pytest, following the existing test patterns in /tests/integration/."
- The Neo analogy from the learnpack title: Neo doesn't learn martial arts by reading a rulebook. He downloads a complete skill — the knowledge, the technique, the context for when to use it. Agent skills work the same way.

01.1 How Skills Work — Triggers, Context, and Execution 📖 (~550 words)

Learning Objectives:

- Describe the mechanism by which a coding agent discovers and activates a skill
- Identify the three core components of a skill: trigger conditions, contextual instructions, and execution guidance
- Explain the relationship between skills and the agent's existing context (rules, memory, codebase)

Content Outline:

- When an agent receives a task, it doesn't just look at rules — it evaluates which skills are relevant to the task at hand. This is similar to how rules use **globs** and **description** fields (students learned this on Day 12), but skills add a richer layer.
- The three parts of how a skill works:
 - **Discovery:** How does the agent know a skill exists? Through skill files in the project, through descriptions that the agent can match against incoming tasks. Similar concept to **alwaysApply**, **Auto Attached**, and **Agent Requested** from rules — but skills tend to be more explicitly task-matched.
 - **Activation:** When the agent determines a skill is relevant, it loads the skill's full context — instructions, constraints, examples, expected outputs. This is where the "Neo downloading kung fu" moment happens.
 - **Execution:** The agent follows the skill's guidance to perform the task, using the skill's context alongside the project's existing rules and memory bank.
- How skills interact with what's already there: a skill doesn't operate in isolation. It works *within* the environment defined by rules and memory. If a rule says "use TypeScript" and a skill describes "how to create an API endpoint," the skill's output respects the rule. They're complementary layers.
- Brief mention: the skill file itself is typically a markdown document with structured sections — but the detailed anatomy of skill files is covered in the next learnpack (Creación de Skills). For now, focus on understanding the mechanism.

Transitions: Students now understand what skills are and how they work mechanically. Next section shifts to the practical question: how do you actually give your agent a new skill?

Key Principles:

- Skills are discovered, activated, and executed — they're not passive configuration
- A skill operates within the existing context (rules + memory + codebase), not independently
- The agent matches tasks to skills based on descriptions and trigger conditions

Examples:

- An agent receives "create a new user endpoint." It discovers a skill called "API endpoint scaffolding" whose description matches. It activates the skill, loading instructions that say: "Create a router file in /api/routes/, a Pydantic model in /models/, and a test file in /tests/. Follow the existing patterns in these directories." It then executes, producing files that also respect the project's formatting rules and TypeScript requirement.
- Contrast with no skill: the same agent with only rules would create an endpoint, but might put files in wrong directories, skip the test stub, or use a different pattern than the rest of the codebase.

Section 02: Teaching Your Agent New Skills

02.0 How to Add Skills to Your Coding Agent 📖 (~550 words)

Learning Objectives:

- Describe the practical steps to add a skill to a coding agent's configuration
- Differentiate between project-level and user-level skill placement
- Identify where skill files live in a typical project structure

Content Outline:

- The practical mechanics: adding a skill is fundamentally about placing a properly structured file where the agent can find it. This varies slightly by tool, but the core concept is consistent.
- Skill file placement:
 - **Project-level skills:** Live in the project repository (e.g., `.cursor/skills/`, `.claude/skills/`, or similar directories depending on the agent). These are shared with the team and versioned with the code.
 - **User-level skills:** Live in the user's global configuration. These are personal preferences and workflows — they travel with you across projects.
- The parallel to rules: students already understand user vs. project rules from Day 12. Skills follow the same scoping logic. A project skill says "this is how we do things here." A user skill says "this is how I like to work."
- What happens when you add a skill: the agent indexes it, reads the description, and makes it available for task matching. The skill doesn't activate until relevant — it's not consuming tokens or attention when unused.
- Important nuance: adding a skill file is necessary but not sufficient. The skill needs to be well-written to be effective. (Brief mention — the next learnpack covers skill creation in depth.)

Transitions: Knowing where to place a skill file is step one. But the agent doesn't just read files — it needs to understand them. Next lesson explores what the agent actually needs from a skill to learn it effectively.

Key Principles:

- Adding a skill = placing a structured file where the agent can discover it
- Project vs. user scoping for skills mirrors the same distinction in rules
- A skill file's existence doesn't guarantee effectiveness — quality matters

Examples:

- A team working on a Next.js project adds a skill file at `.cursor/skills/create-page.md` that describes how to scaffold a new page following their conventions. Every team member's agent now has this capability.
- A developer adds a personal skill to their global config that describes their preferred commit message format. This skill applies across all their projects without polluting any specific repo.

02.1 What Your Agent Needs to Actually Learn a Skill 📖 (~550 words)

Learning Objectives:

- Explain the concept of context engineering as it applies to agent skills

- Identify what information an agent needs to effectively execute a skill
- Recognize the difference between giving an agent instructions and giving it *understanding*

Content Outline:

- The "teaching" metaphor in the learnpack title isn't accidental. Adding a skill file is like handing someone a manual. Whether they can actually *perform* the skill depends on the quality of that manual and the context surrounding it.
- **Context engineering** (reference: Anthropic's article on effective context engineering for AI agents): the idea that the most impactful lever you have over AI behavior is not the model itself but the context you provide. A skill is a form of curated context — it packages exactly the information the agent needs to perform a specific task well.
- What the agent needs to learn a skill effectively:
 - **Clear purpose:** What is this skill for? When should it activate? A vague description means the agent can't match it to the right tasks.
 - **Sufficient context:** The instructions, constraints, and examples needed to execute. Not more, not less — over-stuffed skills waste tokens and dilute attention.
 - **Connection to the codebase:** The agent doesn't learn the skill in a vacuum. It learns it in the context of *this* project, with *this* code structure, following *these* rules. Your codebase is part of the skill's context whether you planned for it or not.
- The "curated and concrete" principle from the Thinking Framework: skills should contain curated, concrete content — not ambiguities or rambling descriptions. Every sentence in a skill file should earn its place.
- Brief connection to memory banks: a well-maintained memory bank (covered earlier today) gives the agent project context that makes skills more effective. Skills and memory are complementary — memory provides the "what we've done and decided," skills provide the "how to do the next thing."

Transitions: Students now understand how to add skills and what makes the agent actually learn them. But before building custom skills (next learnpack), there's a faster path: using skills that already exist. Next section covers the skills ecosystem.

Key Principles:

- Context engineering is the art of giving AI the right information at the right time — skills are a primary vehicle for this
- A skill's effectiveness depends on clarity of purpose, sufficiency of context, and connection to the existing codebase
- More context is not better context — curation matters more than volume

Examples:

- A skill that says "create API endpoints" with no further context vs. a skill that says "create a FastAPI endpoint following the patterns in `/api/routes/users.py`, using Pydantic models from `/models/`, with a corresponding test in `/tests/integration/`" — same instruction, vastly different results.
- Reference to Anthropic's context engineering article: the observation that agents perform best when their context is carefully engineered, not when they're given everything and asked to figure it out.

Section 03: Finding and Using Pre-Built Skills

03.0 Browsing, Evaluating, and Installing Skills from the Ecosystem 📖 (~550 words)

Learning Objectives:

- Navigate skills.sh and similar skill repositories to find relevant pre-built skills
- Apply evaluation criteria to determine whether a pre-built skill is suitable for a project
- Describe the process of installing and adapting a community skill

Content Outline:

- The path of least resistance principle: before writing anything custom, check if someone has already solved your problem. The skills ecosystem exists precisely for this purpose.
- **skills.sh** — the primary skill marketplace/repository. Walk through what students will find there:
 - Skills organized by category and use case
 - Descriptions, usage instructions, and compatibility information
 - Community ratings and feedback
- How to evaluate a skill before installing it:
 - **Does it match your use case?** Read the description carefully. A skill called "API scaffolding" might assume Express.js when you're using FastAPI.
 - **Is it well-scoped?** A skill that tries to do too many things is a red flag (this connects to the "God Skill" anti-pattern from the next learnpack's framework, but at an intuitive level students can already judge this).
 - **Does it conflict with your existing setup?** A skill that enforces conventions different from your project's existing rules and patterns will create confusion, not capability.
 - **Is it maintained?** Community skills can go stale. Check dates and activity.
- Installing a skill: typically involves downloading the skill file and placing it in the appropriate directory (project-level or user-level, as covered in 02.0). Some ecosystems support direct installation commands.
- Adapting vs. using as-is: most pre-built skills benefit from light customization to match your project's specific conventions. This is expected and encouraged — a skill is a starting point, not a sacred text.

Transitions: Students can now find and install skills. But skills don't operate in isolation — they work alongside the codebase. Next lesson explores a critical insight: the quality of your code directly affects how well skills perform.

Key Principles:

- Use what exists before building from scratch (path of least resistance)
- Evaluate skills against your specific project context, not just their description
- Pre-built skills are starting points — adaptation is normal and healthy

Examples:

- A student needs a skill for generating React components. They browse skills.sh, find one for "React component scaffolding," but it assumes CSS Modules while their project uses Tailwind. They install it and adjust the styling instructions — faster than writing from scratch.
- A student finds a highly-rated "database migration" skill but it's built for Prisma while they use SQLAlchemy. This is a mismatch — better to keep looking or build custom.

03.1 Your Code Is Context Too — Why Codebase Quality Affects Skills 📖 (~550 words)

Learning Objectives:

- Explain how the existing codebase acts as implicit context for agent skills
- Recognize that good coding practices amplify skill effectiveness while bad practices compound errors
- Identify specific ways codebase quality influences skill outcomes

Content Outline:

- A critical insight from the Thinking Framework: "el código también es contexto y que las buenas prácticas se replican y que las malas prácticas de un código también se potencian con la IA."
- When an agent executes a skill — say, "create a new API endpoint following existing patterns" — it looks at your *existing code* to understand what "existing patterns" means. If your existing endpoints are clean, consistent, and well-structured, the agent produces clean output. If they're messy, inconsistent, or have accumulated bad patterns, the agent replicates and amplifies that mess.
- This applies to skills specifically because skills often reference existing code:
 - "Follow the pattern in /api/routes/" — the agent reads those files as context
 - "Match the testing approach in /tests/" — the agent inherits whatever quality (or lack thereof) exists there
 - "Use the project's component structure" — if the structure is chaotic, the skill produces chaotic output
- The amplification effect: without AI, bad code is a localized problem. With AI + skills, bad code becomes a *template* that gets replicated at scale. Every new file the skill generates inherits the debt.
- The positive side: teams with clean, consistent codebases get disproportionate value from skills. The agent can reliably follow patterns because the patterns are clear. Skills become genuinely powerful when the codebase they operate on is well-maintained.
- Connection to memory banks and rules: this is why the combination matters. Rules enforce standards. Memory banks track decisions and conventions. Skills package capabilities. And the codebase itself is the ground truth that all three operate on. If any layer is neglected, the others suffer.

Transitions: Students now understand skills from every angle — what they are, how to add them, where to find them, and how codebase quality affects them. The final content section zooms out to place skills in the broader context of the tools students have learned so far, and addresses common mistakes.

Key Principles:

- Your codebase is implicit context for every skill — treat it as such
- Good code + good skills = amplified productivity. Bad code + good skills = amplified debt.
- Skills, rules, memory, and code quality are four interdependent layers — neglecting any one weakens the others

Examples:

- Two teams install the same "API endpoint" skill. Team A has consistent route files with clear naming, standard error handling, and typed models. Team B has mixed patterns, some endpoints with validation, some without, inconsistent naming. The skill produces excellent results for Team A and messy, inconsistent results for Team B — same skill, different codebases.
- A student adds a "component generator" skill to a React project where half the components use class syntax and half use hooks. The skill doesn't know which pattern to follow, so it picks whichever it encounters first — potentially creating more inconsistency.

Section 04: Skills in Context

04.0 Skills vs. Rules vs. Memory — Choosing the Right Tool 📖 (~550 words)

Learning Objectives:

- Compare and contrast skills, rules, and memory banks across specific dimensions (purpose, scope, persistence, activation)
- Apply a decision framework to determine which tool is appropriate for a given situation
- Recognize that the three tools are complementary layers, not competing alternatives

Content Outline:

- Students now know three tools for shaping agent behavior. The challenge is knowing when to use which. This lesson provides a clear comparison framework.
- Comparison across dimensions:
 - **Purpose:** Rules constrain behavior ("always do X, never do Y"). Memory provides persistent context ("here's what we've decided, here's the project state"). Skills package capabilities ("here's how to do X").
 - **Scope:** Rules can be global or file-pattern-specific (globs). Memory is project-wide context. Skills are task-specific — they activate when relevant and stay dormant otherwise.
 - **Persistence:** Rules are static until changed. Memory evolves as the project progresses. Skills are stable blueprints that produce different outputs depending on context.
 - **Activation:** Rules are always active or pattern-matched. Memory is always available as background context. Skills are triggered by task relevance.
- Decision framework — when faced with a new need, ask:
 - "Is this about what the agent should *always* or *never* do?" → **Rule**
 - "Is this knowledge the agent needs to *remember* across sessions?" → **Memory**
 - "Is this a *capability* the agent should have for specific tasks?" → **Skill**
- Common confusion points:
 - A long, detailed rule that describes *how* to do something is probably a skill trying to be a rule.
 - A skill that applies to every single task regardless of context is probably a rule trying to be a skill.
 - Memory that contains step-by-step instructions is probably a skill stored in the wrong place.
- The layered model: rules form the base layer (constraints), memory provides the middle layer (context), skills sit on top (capabilities). A well-configured agent uses all three in harmony.

Transitions: With a clear understanding of when to use each tool, the next lesson addresses what goes wrong — common mistakes students should avoid when working with agent skills.

Key Principles:

- Rules constrain, memory contextualizes, skills enable — each serves a distinct function
- Misplacing content (a skill disguised as a rule, memory stored as a skill) creates confusion for the agent
- The three tools are layers that work together — the goal is harmony, not choosing just one

Examples:

- "Use snake_case for all Python variables" → **Rule** (universal constraint, always active)

- "We decided in Sprint 3 to use PostgreSQL instead of SQLite, and the migration is tracked in /docs/adr-003.md" → **Memory** (project decision, persistent context)
 - "How to create a new database migration: generate a file in /migrations/ using Alembic, following the naming pattern YYYYMMDD_description.py, with upgrade and downgrade functions" → **Skill** (specific capability, task-triggered)
 - Edge case: "Always run linting before committing" — this could be a rule OR a skill. As a rule, it's a simple constraint. As a skill, it could include the exact linting command, how to handle errors, and how to auto-fix common issues. The choice depends on how much guidance the agent needs.
-

04.1 Common Mistakes When Working with Agent Skills 📖 (~550 words)

Learning Objectives:

- Identify the most common anti-patterns when using agent skills
- Explain the consequences of each anti-pattern
- Apply the correct approach as an alternative to each mistake

Content Outline:

- **Blind trust in agent proactivity:** Students assume that because they added a skill, the agent will always use it perfectly. Reality: the agent matches skills based on descriptions and context. If the description is vague or the task framing doesn't match, the skill won't activate — or worse, the wrong skill activates. Correct approach: verify that the agent is using the right skill by reviewing its reasoning/output, especially when first introducing a skill.
- **Overwriting existing project rules with personal preferences:** A student joins a project with established conventions and adds skills that contradict them — not because the existing approach is wrong, but because they prefer something different. This creates conflict between skills and rules, confusing the agent. Correct approach: respect existing project configuration. Propose changes through team discussion, don't unilaterally override via personal skills.
- **Ambiguous, overly descriptive skill definitions:** A skill that tries to explain everything in paragraph form, full of qualifiers and edge cases, instead of being concrete and structured. The agent's attention is diluted, and it struggles to extract the actionable parts. Correct approach: curated, concrete content. Every line in a skill should earn its place.
- **Pasting long conversations as skill content:** Students copy entire chat histories or long discussions into skill files, thinking more context is better. This overwhelms the agent's context window and buries the useful information. Correct approach: synthesize conversations into clear, structured skill content — extract the decisions, discard the discussion.
- **Creating a purely technical context and omitting the business:** A skill that describes *how* to build something but never *why* or *for whom*. The agent can follow steps but can't make good judgment calls when the steps aren't enough. Correct approach: include minimal but sufficient business context — what problem this skill solves and for whom.

Transitions: Students now have a complete picture: what skills are, how to add and find them, how codebase quality matters, when to use skills vs. rules vs. memory, and what mistakes to avoid. The assessment will test their understanding across all these dimensions.

Key Principles:

- Trust but verify — adding a skill doesn't guarantee correct activation
- Respect existing conventions — skills should extend a project's configuration, not contradict it
- Concrete over verbose — a well-structured short skill outperforms a rambling long one

Examples:

- Anti-pattern: A skill file with a 500-word description that starts with "This skill is designed to help the agent understand the nuanced process of..." — the agent loses the signal in the noise. Fixed version: a 50-word description that states exactly what the skill does and when it applies.
 - Anti-pattern: A developer adds a personal skill that enforces tabs for indentation in a project that uses spaces. The agent now receives conflicting instructions and produces inconsistent output. Fixed: the developer discusses the change with the team or keeps personal preferences separate from project skills.
 - Anti-pattern: A skill whose entire content is a pasted Slack thread about how the team debugged a deployment issue. The agent can't extract actionable guidance from conversational text. Fixed: the developer synthesizes the thread into a structured skill — "How to debug deployment failures in our CI/CD pipeline: 1) Check the build logs at..., 2) Verify the environment variables..."
-

04.2 Agent Skills Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of agent skills — definition, mechanics, and purpose
- Distinguish between appropriate uses of skills, rules, and memory banks
- Identify correct and incorrect approaches to working with skills

Question Format: Scenario-based (all questions)

Questions:

Q1: Your coding agent keeps formatting code inconsistently across the project — sometimes using single quotes, sometimes double quotes. You want to fix this. Should you create a skill, add a rule, or update the memory bank?

Expected reasoning: This is a universal constraint (always use single quotes OR double quotes). It's a **rule**, not a skill. A skill would be overkill for a simple formatting preference. Memory bank is for project decisions and context, not behavioral constraints.

Q2: You join an existing project where the team has established skills for API endpoint creation, component scaffolding, and test generation. You prefer a different testing approach (Jest instead of the project's Vitest setup). What should you do?

Expected reasoning: Respect the existing project configuration. The correct approach is to discuss the change with the team, not to override the existing skill with a personal one. Overwriting project conventions with personal preferences is an anti-pattern.

Q3: A teammate adds a skill file that contains a 3-page narrative about the history of the project's architecture, followed by two lines of actual instructions. The agent doesn't seem to follow the skill consistently. What's likely wrong?

Expected reasoning: The skill is too verbose and buries the actionable content in narrative. The agent's attention is diluted. The fix is to synthesize the skill into curated, concrete instructions — keeping only what the agent needs to execute, possibly moving the architectural history to the memory bank where it belongs.

Q4: You find a skill on skills.sh called "React Component Generator" that has high ratings. Your project uses React but with a custom design system and Tailwind CSS, while the skill assumes Material UI. Should you install it as-is?

Expected reasoning: No — the skill's assumptions conflict with the project's stack. The student should either look for a more compatible skill, or install this one and adapt it to reference Tailwind and the custom design system instead of Material UI. Pre-built skills are starting points, not sacred texts.

Q5: Your project has clean, well-organized code with consistent patterns. Your teammate's project has accumulated technical debt — inconsistent naming, mixed patterns, no clear structure. Both of you install the same "API scaffolding" skill. Why might you get better results?

Expected reasoning: The codebase is implicit context for skills. When the skill says "follow existing patterns," it reads the actual code. Clean, consistent patterns give the agent clear guidance. Messy, inconsistent code gives the agent contradictory signals. Good code amplifies skill effectiveness; bad code amplifies problems.

Q6: You're trying to decide whether a piece of information should be a skill or a memory bank entry. The information is: "In Sprint 4, we decided to switch from REST to GraphQL for all new endpoints. The migration plan is in /docs/adr-005.md." Where does this belong?

Expected reasoning: This is a project decision that provides persistent context — it's **memory bank** content. It's not a capability (skill) because it doesn't describe *how* to create GraphQL endpoints. A complementary skill might be "How to create a new GraphQL endpoint following our conventions," but the decision itself is memory.

Evaluation Criteria:

- Each answer is evaluated on reasoning quality, not just the final choice
- Students should demonstrate understanding of *why* one tool is appropriate over another
- Partial credit for answers that identify the right tool but with weak justification

Score Interpretation:

- 6/6: Strong grasp of agent skills and their relationship to the broader context engineering toolkit
- 4-5/6: Good understanding with some gaps in distinguishing between tools or identifying anti-patterns
- 3 or below: Review Sections 01-04, particularly the decision framework in 04.0 and anti-patterns in 04.1

Section 05: Conclusion

05.0 Your Agent's Growing Toolbox (~450 words)

Content Outline:

- Course recap: what students accomplished
 - Understood what agent skills are and how they differ from rules and memory
 - Learned the mechanics of how agents discover, activate, and execute skills

- Explored how to add skills to a coding agent and what the agent needs to learn effectively
 - Discovered the skills ecosystem (skills.sh) and how to evaluate pre-built skills
 - Recognized that codebase quality directly affects skill performance
 - Built a decision framework for choosing between skills, rules, and memory
 - Identified common mistakes and how to avoid them
- Connection to bigger picture: skills are one layer in a broader context engineering strategy. Students now have three powerful tools — rules, memory, and skills — each serving a distinct purpose. The agent's effectiveness comes from how well these layers work together, not from any single one in isolation.
- Next steps: the next learnpack ("Creación de Skills") will take students from consumers to creators — learning how to design, structure, test, and compose custom skills from scratch. Everything learned here about what makes skills effective becomes the foundation for building them.
- Resources for further exploration:
 - skills.sh — browse and discover community skills
 - Anthropic's article on effective context engineering for AI agents (<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>)
 - The agent's own documentation for skill configuration (Cursor, Claude, Windsurf, etc.)
- Encouragement: you've gone from writing raw prompts to configuring rules, building memory banks, and now understanding skills. Your agent is becoming a genuinely capable collaborator — and you're becoming the kind of developer who knows how to make that collaboration effective.

Note: This is conclusion only — no theory, exercises, or assessment.
